Deployment Instructions For Portfolio-2

Assume your domain is yourdomain.com. Replace it everywhere with your real Namecheap domain.

0. Understand What You Are Deploying
This repo has multiple parts:
- apps/web: public portfolio website
- apps/admin: private admin dashboard
- services/api: FastAPI backend
- services/worker: background jobs
- Postgres: database
- Redis: cache, rate limit, cost logs
- Namecheap: domain/DNS only

Namecheap is not where this project should run. Namecheap will only point your domain to Vercel and Render.

1. Create Accounts
Create or login to these accounts:
1. GitHub: for code repo
2. Vercel: for public website and admin dashboard
3. Render: for FastAPI backend
4. Neon: for free Postgres database
5. Upstash: for free Redis
6. Resend: for admin magic-link email login
7. Namecheap: your domain DNS

2. Fix Secret Ignore Safety First
Open terminal in the project:
cd /Users/quantum-pulse/Important/Websites/Portfolio-2

Open .gitignore and make sure it contains:
.env
.env.local
.env.*.local
infra/.env.production

Your .gitignore already ignores .env, but add infra/.env.production if missing. This prevents accidentally committing production secrets.

3. Check There Is No Git Repo Yet
Run:
git status

If it says fatal: not a git repository, that is okay. It means you still need to create a Git repo.

4. Scan For Real Secrets Before GitHub
Run:
rg -n --hidden --glob '!.git/**' --glob '!node_modules/**' --glob '!*.lock'
"(OPENAI_API_KEY|ANTHROPIC_API_KEY|DATABASE_URL=|REDIS_URL=|ADMIN_TOKEN=|AUTH_SECRET=|RESEND_

Look at every result.
Allowed results: OPENAI_API_KEY=, change-me, example.com.
Not allowed: sk-..., postgresql://real-user:real-password@..., re_..., real token values.
If you see real secrets in a file, remove them before pushing.

5. Create GitHub Repo
From project root:
git init
git add .
git status

Check that .env or infra/.env.production is not listed.
Then commit:
git commit -m "Initial portfolio platform"

Create a new GitHub repo from GitHub website.
Then connect your local repo:
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO_NAME.git
git branch -M main
git push -u origin main

6. Enable GitHub Secret Protection
In GitHub:
1. Open your repo
2. Go to Settings
3. Go to Code security
4. Enable secret scanning if available
5. Enable push protection if available

7. Create Neon Postgres
Go to Neon:
1. Create new project
2. Choose free plan
3. Name it engine-room
4. Choose nearest region
5. Open connection string
6. Copy the pooled or direct connection string

It may look like:
postgresql://user:password@host.neon.tech/dbname?sslmode=require

For this FastAPI app, save it as:
postgresql+asyncpg://user:password@host.neon.tech/dbname?ssl=require

Important: replace postgresql:// with postgresql+asyncpg://.

8. Create Upstash Redis
Go to Upstash:
1. Create Redis database
2. Choose free plan

3. Choose nearest region
4. Copy the Redis URL

It should look like:
rediss://default:password@host.upstash.io:6379

Save it as REDIS_URL.

9. Generate Your Private Tokens
In terminal:
openssl rand -base64 32
Copy the output. This is your ADMIN_TOKEN.

Run again:
openssl rand -base64 32
Copy the output. This is your AUTH_SECRET.

Keep them somewhere safe, like a password manager. Do not put them in code.

10. Create Resend API Key
Go to Resend:
1. Create account
2. Add your sending domain if possible
3. Create API key
4. Copy API key

It looks like re_...
Use it as RESEND_API_KEY.

11. Deploy API To Render
Go to Render:
1. Click New
2. Click Web Service
3. Connect GitHub
4. Select your repo
5. Name it engine-room-api
6. Runtime: Python
7. Root directory: leave blank / repo root
8. Build command: pip install uv && uv sync --all-packages --all-extras
9. Start command: cd services/api && uv run --package engine-room-api alembic upgrade head && cd ../.. && uv run --package engine-room-api uvicorn app.main:app --app-dir services/api --host 0.0.0.0 --port $PORT
10. Health check path: /api/health
11. Choose free instance type

12. Add Render API Environment Variables
Add these in Render service environment:
ENVIRONMENT=production
PYTHON_VERSION=3.12.8
DATABASE_URL=postgresql+asyncpg://YOUR_NEON_URL
REDIS_URL=rediss://YOUR_UPSTASH_URL
API_CORS_ORIGINS=["https://yourdomain.com","https://www.yourdomain.com","https://admin.yourdo

ADMIN_TOKEN=YOUR_RANDOM_ADMIN_TOKEN
DEFAULT_FLAGS={}
OPENAI_API_KEY=
ANTHROPIC_API_KEY=
EMBEDDING_MODEL=text-embedding-3-small
SUBSTACK_FEED_URL=https://your-substack.substack.com/feed

If you do not have OpenAI or Anthropic keys yet, leave them empty.

13. Get Render API URL
After deploy, Render gives you a URL like:
https://engine-room-api.onrender.com

Open:
https://engine-room-api.onrender.com/api/health

Expected: status ok, db true, redis true. If db is false, DATABASE_URL is wrong. If redis is false, REDIS_URL is wrong.

14. Deploy Public Website To Vercel
Go to Vercel:
1. Add New
2. Project
3. Import your GitHub repo
4. Framework should detect Next.js
5. Root Directory: apps/web
6. Build command: pnpm build
7. Install command: pnpm install
8. Add environment variables:
NEXT_PUBLIC_SITE_URL=https://yourdomain.com
NEXT_PUBLIC_API_BASE_URL=https://engine-room-api.onrender.com
API_INTERNAL_BASE_URL=https://engine-room-api.onrender.com
9. Deploy

15. Deploy Admin App To Vercel
Create another Vercel project from the same repo:
1. Root Directory: apps/admin
2. Add environment variables:
AUTH_SECRET=YOUR_RANDOM_AUTH_SECRET
AUTH_URL=https://admin.yourdomain.com
AUTH_TRUST_HOST=true
ADMIN_EMAIL=you@example.com
ADMIN_TOKEN=THE_SAME_ADMIN_TOKEN_USED_IN_RENDER_API
RESEND_API_KEY=YOUR_RESEND_API_KEY
EMAIL_FROM=admin@yourdomain.com
NEXT_PUBLIC_API_BASE_URL=https://engine-room-api.onrender.com
API_INTERNAL_BASE_URL=https://engine-room-api.onrender.com

ADMIN_TOKEN must be exactly the same in Render API and Vercel Admin.

16. Seed Production Database
From your local terminal, run this using the Neon URL. Use normal Postgres URL here, not

asyncpg:
DATABASE_URL='postgresql://user:password@host.neon.tech/dbname?sslmode=require'
ADMIN_EMAIL='you@example.com' uv run python infra/seed.py

17. Test API After Seed
Run:
curl -fsS https://engine-room-api.onrender.com/api/health
curl -fsS https://engine-room-api.onrender.com/api/projects
curl -fsS https://engine-room-api.onrender.com/api/now

18. Test Public Website
Open your Vercel public URL. Check homepage, projects, search, contact form, and resume behavior.
If data is missing, check NEXT_PUBLIC_API_BASE_URL and API_INTERNAL_BASE_URL.

19. Test Admin Dashboard
Open https://your-admin.vercel.app/sign-in. Enter your ADMIN_EMAIL. Click the email magic link. Try editing a project.
If login works but edits fail, ADMIN_TOKEN is mismatched between Vercel admin and Render API.

20. Add Custom Domain In Vercel For Public Site
In Vercel public project:
1. Settings
2. Domains
3. Add yourdomain.com
4. Add www.yourdomain.com

Vercel usually asks for:
@ A 76.76.21.21
www CNAME cname.vercel-dns.com
Use exactly what Vercel shows.

21. Add Custom Domain In Vercel For Admin
In Vercel admin project:
1. Settings
2. Domains
3. Add admin.yourdomain.com

Vercel usually asks for:
admin CNAME cname.vercel-dns.com
Use exactly what Vercel shows.

22. Add Custom Domain In Render For API
In Render API service:
1. Settings
2. Custom Domains
3. Add api.yourdomain.com

Render will show a DNS target. Use exactly what Render shows.

23. Set DNS In Namecheap

In Namecheap:
1. Login
2. Domain List
3. Manage beside your domain
4. Advanced DNS
5. Host Records
6. Delete conflicting old records for @, www, admin, api
7. Add records:

A Record: Host @, Value 76.76.21.21, TTL Automatic
CNAME Record: Host www, Value cname.vercel-dns.com, TTL Automatic
CNAME Record: Host admin, Value cname.vercel-dns.com, TTL Automatic
CNAME Record: Host api, Value YOUR_RENDER_DNS_TARGET, TTL Automatic

Save all changes. DNS can take 5 minutes to a few hours.

24. Update Environment Variables To Custom Domains
Update Render API:
API_CORS_ORIGINS=["https://yourdomain.com","https://www.yourdomain.com","https://admin.yourdo

Update Vercel public app:
NEXT_PUBLIC_SITE_URL=https://yourdomain.com
NEXT_PUBLIC_API_BASE_URL=https://api.yourdomain.com
API_INTERNAL_BASE_URL=https://api.yourdomain.com

Update Vercel admin app:
AUTH_URL=https://admin.yourdomain.com
NEXT_PUBLIC_API_BASE_URL=https://api.yourdomain.com
API_INTERNAL_BASE_URL=https://api.yourdomain.com

Redeploy Render API, Vercel public web, and Vercel admin.

25. Final Verification Commands
Run:
curl -fsS https://api.yourdomain.com/api/health
curl -fsSI https://yourdomain.com
curl -fsSI https://www.yourdomain.com
curl -fsSI https://admin.yourdomain.com/sign-in
curl -fsS https://api.yourdomain.com/api/projects
curl -fsS -X POST https://api.yourdomain.com/api/search -H "content-type:
application/json" -d '{"query":"projects"}'

26. Browser Verification
Open https://yourdomain.com and check homepage, images, projects, search, contact form, and browser console.
Open https://admin.yourdomain.com/sign-in and check magic link login, admin pages, editing, and public update.
Open https://api.yourdomain.com/docs and check FastAPI docs.

27. Never Save These In Code
Never commit:
DATABASE_URL

REDIS_URL
ADMIN_TOKEN
AUTH_SECRET
RESEND_API_KEY
OPENAI_API_KEY
ANTHROPIC_API_KEY
AWS_SECRET_ACCESS_KEY

Only these are safe to expose if intended:
NEXT_PUBLIC_SITE_URL
NEXT_PUBLIC_API_BASE_URL

Anything starting with NEXT_PUBLIC_ can be seen in the browser.

28. Where To Store Secrets
Store secrets only in Render Environment Variables, Vercel Environment Variables, Neon dashboard, Upstash dashboard, Resend dashboard, and a password manager.
Do not store secrets in README.md, committed .env files, public Notion pages, screenshots, chat messages, client-side code, or NEXT_PUBLIC_* variables.

29. Secret Leak Check Before Every Push
Before pushing:
git status --short
rg -n --hidden --glob '!.git/**' --glob '!node_modules/**' --glob '!*.lock'
"(sk-|re_|postgresql://|postgresql\\+asyncpg://|rediss://|ADMIN_TOKEN=|AUTH_SECRET=|RESEND_AP

If real secrets appear, stop and remove them.
Then:
git add .
git commit -m "Your message"
git push

30. If A Secret Leaks
Do not just delete it from Git.
1. Rotate the leaked secret in the provider dashboard
2. Replace it in Render/Vercel
3. Redeploy affected services
4. Remove it from the repo
5. Consider cleaning Git history if the repo is public

If ADMIN_TOKEN leaks:
openssl rand -base64 32
Then update Render API ADMIN_TOKEN and Vercel admin ADMIN_TOKEN, redeploy both, and test admin edits again.

31. Free Plan Limits To Remember
Render free API sleeps when unused. First request may be slow.
Vercel free is good for portfolio frontend.
Neon free can pause or scale down depending on usage.
Upstash free has request limits.
This is fine for a personal portfolio. For real production, move API/database to paid plans.

32. Minimal Working Checklist
You are done only when all are true:
- GitHub repo has no secrets
- Render API health works
- Neon database migrated
- Seed script ran successfully
- Upstash Redis works
- Vercel public site loads
- Vercel admin login works
- Namecheap points your domain correctly
- https://yourdomain.com works
- https://admin.yourdomain.com works
- https://api.yourdomain.com/api/health works
- Admin edits appear on public site